

GOVERNMENT COLLEGE OF ENGINEERING

DHARMAPURI – 636 704

(Affiliated to Anna University, Chennai)



BONAFIDE CERTIFICATE

NAME :

REG.NO :

DEGREE & BRANCH : B.E & ECE

SUBJECT CODE & TITLE : NM1135 & IoT Application (ESP32)

YEAR & SEM : III & VI

Certified that this Bonafide Record of Work done by the above student of

the..... Laboratory

during the year 2026- 2027.

Faculty In-Charge

Head of the Department

Submitted for the University Practical Examination held on 12.05.2026 at

Government College of Engineering, Dharmapuri.

Internal Examiner

External Examiner

Contents

Sl.no	Date	Name of the Experiment	Page no	Marks	Staff Sign
1.	7/1/26	Introduction to IoT & ESP32 – LED Blink Experiment	2		
2.	21/1/26	ESP32 LED Control Using Push Button with Debouncing	4		
3.	28/1/26	Sensor Interfacing & Real-Time Data Acquisition	8		
4.	4/2/26	Actuator & Display Handling Using ESP32	11		
5.	11/2/26	Communication Protocols Using ESP32	15		
6.	18/2/26	MQTT & Networking using ESP32	18		
7.	25/2/26	Cloud Connectivity using ESP32	22		
8.	4/3/26	Database & Google Sheets Logging using ESP32	25		
9.	11/3/26	Multi-Node IoT Systems using ESP32	29		
10	11/3/26	IoT Security & Optimization using ESP32	34		
11	18/3/26	Real-World IoT Applications	38		
12	18/3/26	Final Project & Evaluation	43		

Exp No 1: Introduction to IoT & ESP32 – LED Blink Experiment

Aim

To study the basics of IoT and ESP32 and perform LED blinking using GPIO control.

Apparatus Required

- ESP32 Development Board
- Breadboard
- LED
- 220Ω Resistor
- Jumper Wires
- USB Cable
- Arduino IDE

Theory

The Internet of Things (IoT) is a network of interconnected devices capable of sensing, processing, and exchanging information automatically. ESP32 is a microcontroller widely used in IoT applications because it supports Wi-Fi, Bluetooth, GPIO, ADC, DAC, PWM, and timers.

GPIO pins allow the ESP32 to interact with external devices such as LEDs and sensors. The LED blink experiment demonstrates basic digital output operation using HIGH and LOW logic signals.

Circuit Connections

Component ESP32 Pin

LED Positive GPIO 2

LED Negative GND through 220Ω resistor

Program

```
const int ledPin = 2;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  delay(1000);

  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

Procedure

1. Connect the LED to GPIO pin 2 through a resistor.
2. Connect ESP32 to the computer using USB cable.
3. Open Arduino IDE.
4. Select ESP32 board and COM port.
5. Enter the program and compile it.
6. Upload the code to ESP32.
7. Observe the LED blinking continuously.

Output

- LED blinks ON and OFF repeatedly at 1-second interval.

Result

Thus, the LED blink experiment using ESP32 GPIO control was successfully implemented.

Viva Questions

1. What is GPIO?
2. Why is debouncing necessary?
3. Difference between INPUT and INPUT_PULLUP?
4. What is PWM?
5. Why are resistors used with LEDs?

Exp No 2: ESP32 LED Control Using Push Button with Debouncing

Aim

To interface a push button with ESP32 and control an LED using software debouncing.

Apparatus Required

- ESP32 Development Board
- Breadboard
- LED
- Push Button
- 220Ω Resistor
- Jumper Wires
- USB Cable

Theory

Embedded programming involves writing software for hardware systems like microcontrollers. The ESP32 supports digital input/output operations for controlling external devices.

A push button may produce unwanted rapid ON/OFF transitions called **mechanical bouncing**. Debouncing is used to eliminate false triggering and ensure stable operation.

Circuit Connections

Component	ESP32 Pin
LED Positive	GPIO 2
LED Negative	GND through 220Ω
Push Button	GPIO 4
Other Button Terminal	GND

Use internal pull-up resistor in code.

Program

```
const int ledPin = 2;
const int buttonPin = 4;

bool ledState = LOW;
bool lastButtonState = HIGH;
bool buttonState;

unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 50;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);

  Serial.begin(115200);
}

void loop() {
  int reading = digitalRead(buttonPin);

  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {

    if (reading != buttonState) {
      buttonState = reading;

      if (buttonState == LOW) {
        ledState = !ledState;
        digitalWrite(ledPin, ledState);

        Serial.println("Button Pressed");
      }
    }
  }
}
```

```
lastButtonState = reading;  
}
```

Procedure

1. Connect the circuit as per diagram.
2. Open Arduino IDE.
3. Select ESP32 board and COM port.
4. Enter the program and compile it.
5. Upload the code to ESP32.
6. Press the push button and observe LED operation.
7. Verify stable switching without false triggering.

Output

- LED toggles ON/OFF when button is pressed.
- Debouncing prevents unstable switching.

Result

Thus, the push button was successfully interfaced with ESP32 and the LED was controlled using software debouncing.

Viva Questions

1. What is IoT?
2. What is ESP32?
3. What is GPIO?
4. Why is a resistor used with LED?
5. What is the function of setup() and loop()?
6. What is PWM?

Exp No 3: Sensor Interfacing & Real-Time Data Acquisition

Aim

To interface sensors with ESP32 and acquire real-time data using Serial Monitor.

Apparatus Required

- ESP32 Development Board
- DHT11 Temperature & Humidity Sensor
- Breadboard
- Jumper Wires
- USB Cable
- Arduino IDE

Theory

Sensor interfacing is the process of connecting sensing devices to a microcontroller for measuring environmental parameters and converting them into digital data. Sensors form the perception layer of IoT systems and are essential for automation and monitoring applications.

The DHT11 sensor measures temperature and humidity using a digital single-wire communication protocol. The ESP32 reads sensor data and displays it through the Serial Monitor for real-time monitoring.

Circuit Connections

DHT11 Pin ESP32 Pin

VCC	3.3V
GND	GND
DATA	GPIO 4

Program

```
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  delay(2000);
}
```

Procedure

1. Connect the DHT11 sensor to ESP32.
2. Open Arduino IDE.
3. Install DHT sensor library if required.
4. Select ESP32 board and COM port.
5. Enter and compile the program.
6. Upload the code to ESP32.

7. Open Serial Monitor.
8. Observe temperature and humidity readings continuously.

Output

- Real-time temperature and humidity values are displayed in Serial Monitor.

Result

Thus, the DHT11 sensor was successfully interfaced with ESP32 and real-time environmental data was acquired.

Viva Questions

1. What is sensor interfacing?
2. What is ADC in ESP32?
3. Difference between analog and digital sensors?
4. What is the function of Serial Monitor?
5. What is calibration?
6. What is real-time data acquisition?

Exp No 4 : Actuator & Display Handling Using ESP32

Aim

To interface an actuator and display module with ESP32 and display real-time information.

Apparatus Required

- ESP32 Development Board
- Servo Motor / DC Motor
- OLED Display (or 16x2 LCD with I2C)
- Breadboard
- Jumper Wires
- USB Cable
- Arduino IDE

Theory

Actuators convert electrical signals into physical actions, while display modules provide visual feedback to users. In IoT systems, actuators and displays form the output layer and help in automation and real-time monitoring.

Servo motors are commonly used for precise angular movement and are controlled using PWM signals from the ESP32. OLED and LCD displays are used to show sensor values and system status information.

Circuit Connections

Servo Motor

Servo Pin ESP32 Pin

VCC 5V

GND GND

Signal GPIO 18

OLED Display (I2C)

OLED Pin ESP32 Pin

VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

Program

```
#include <ESP32Servo.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Servo myServo;

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {

  myServo.attach(18);

  Serial.begin(115200);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED failed");
    while(true);
  }

  display.clearDisplay();
}

void loop() {

  myServo.write(0);
```

```

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(10,20);
display.println("Servo Angle: 0");
display.display();

delay(2000);

myServo.write(90);

display.clearDisplay();
display.setCursor(10,20);
display.println("Servo Angle: 90");
display.display();

delay(2000);
}

```

Procedure

1. Connect the servo motor and OLED display to ESP32.
2. Open Arduino IDE.
3. Install required libraries:
 - ESP32Servo
 - Adafruit SSD1306
 - Adafruit GFX
4. Select ESP32 board and COM port.
5. Enter and compile the program.
6. Upload the code to ESP32.
7. Observe servo rotation and OLED display output.

Output

- Servo motor rotates between 0° and 90°.

- OLED display shows servo angle information.

Result

Thus, the actuator and display modules were successfully interfaced with ESP32 and real-time output was displayed.

Viva Questions

1. What is an actuator?
2. What is PWM?
3. Why is a motor driver required for DC motors?
4. What is the advantage of I2C communication?
5. Difference between LCD and OLED displays?
6. What is a closed-loop control system?

Exp No 5: Communication Protocols Using ESP32

Aim

To study communication protocols and establish Wi-Fi communication using ESP32.

Apparatus Required

- ESP32 Development Board
- USB Cable
- Wi-Fi Network
- Arduino IDE
- Breadboard and Jumper Wires

Theory

Communication protocols define the rules for data exchange between IoT devices, sensors, and cloud systems. Common protocols used in embedded systems include UART, I2C, SPI, and Wi-Fi communication.

UART provides asynchronous serial communication using TX and RX lines. I2C uses two wires (SDA and SCL) for connecting multiple devices, while SPI enables high-speed full-duplex communication. ESP32 also supports Wi-Fi and HTTP protocols for internet-based IoT applications.

Wi-Fi Communication Experiment Using ESP32

Circuit Connections

- No external circuit required for basic Wi-Fi connection.

Program

```
#include <WiFi.h>
```

```
const char* ssid = "YOUR_WIFI_NAME";
```

```
const char* password = "YOUR_WIFI_PASSWORD";
```

```
void setup() {
```



```

Serial.begin(115200);

WiFi.begin(ssid, password);

Serial.print("Connecting to WiFi");

while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.print(".");
}

Serial.println("");
Serial.println("WiFi Connected");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
}

void loop() {

}

```

Procedure

1. Open Arduino IDE.
2. Enter Wi-Fi SSID and password in the program.
3. Select ESP32 board and COM port.
4. Compile and upload the code.
5. Open Serial Monitor with baud rate 115200.
6. Observe Wi-Fi connection status and IP address.

Output

- ESP32 successfully connects to Wi-Fi network.
- IP address is displayed in Serial Monitor.

Result

Thus, communication protocols were studied and Wi-Fi communication using ESP32 was successfully established.

Viva Questions

1. What is a communication protocol?
2. What is UART?
3. Difference between I2C and SPI?
4. What is baud rate?
5. What is HTTP?
6. What is the function of Wi-Fi in IoT?
7. What are SDA and SCL lines in I2C?
8. Why is SPI faster than I2C?

Exp No 6:**MQTT & Networking using ESP32****Aim**

To understand MQTT communication and implement data publishing and subscribing using ESP32 in an IoT network.

Apparatus Required

- ESP32 Development Board
- USB Cable
- Wi-Fi Network
- MQTT Broker (Mosquitto/HiveMQ)
- Arduino IDE
- DHT11 Sensor (optional)

Theory

MQTT (Message Queuing Telemetry Transport) is a lightweight publish–subscribe messaging protocol used in IoT applications. It works efficiently on low-bandwidth networks and allows devices to communicate through a central MQTT broker.

In MQTT:

- Publisher sends data to a topic
- Broker manages message distribution
- Subscriber receives messages from subscribed topics

Example Topic:

iot/temp

QoS Levels:

- QoS 0 – At most once
- QoS 1 – At least once
- QoS 2 – Exactly once

Circuit Diagram

ESP32 connected to computer through USB cable and connected to Wi-Fi network.

Procedure

1. Install Arduino IDE and ESP32 board package.
2. Install PubSubClient library.
3. Connect ESP32 to Wi-Fi.
4. Configure MQTT broker address.
5. Upload MQTT publish/subscribe program.
6. Open Serial Monitor.
7. Observe published and received messages.

Program

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
  delay(10);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void callback(char* topic, byte* message, unsigned int length) {
  Serial.print("Message arrived: ");
```

```

    for (int i = 0; i < length; i++) {
        Serial.print((char)message[i]);
    }
    Serial.println();
}

void reconnect() {
    while (!client.connected()) {
        if (client.connect("ESP32Client")) {
            client.subscribe("iot/test");
        } else {
            delay(5000);
        }
    }
}

void setup() {
    Serial.begin(115200);
    setup_wifi();

    client.setServer(mqtt_server, 1883);
    client.setCallback(callback);
}

void loop() {
    if (!client.connected()) {
        reconnect();
    }

    client.loop();

    client.publish("iot/test", "Hello from ESP32");
    delay(2000);
}

```

Observation

- ESP32 successfully connected to Wi-Fi.
- Messages were published to MQTT broker.
- Subscribed topic received messages correctly.

Result

Thus, MQTT communication between ESP32 and MQTT broker was successfully implemented using publish–subscribe architecture.

Exp No 7:

Cloud Connectivity using ESP32

Aim

To connect ESP32 with a cloud platform and upload sensor data for remote monitoring and control.

Apparatus Required

- ESP32 Development Board
- USB Cable
- Wi-Fi Network
- Arduino IDE
- Cloud Platform (Blynk / ThingsBoard / Adafruit IO)
- DHT11 Sensor (optional)

Theory

Cloud connectivity in IoT enables devices to send sensor data to remote cloud servers for storage, visualization, analytics, and remote control. Cloud platforms provide dashboards, alerts, and automation features for IoT applications.

Popular cloud platforms:

- Blynk
- ThingsBoard
- Adafruit IO

Advantages:

- Remote monitoring
- Data storage
- Device control
- Real-time visualization

Circuit Diagram

ESP32 connected to computer via USB and connected to Wi-Fi network.

Procedure

1. Install Arduino IDE and ESP32 board package.
2. Create an account in Blynk cloud platform.
3. Create a new template and obtain Auth Token.
4. Install Blynk library in Arduino IDE.
5. Connect ESP32 to Wi-Fi.
6. Upload the cloud connectivity program.
7. Open Blynk dashboard and observe sensor data.

Program

```
#define BLYNK_TEMPLATE_ID "TMPLxxxx"
#define BLYNK_DEVICE_NAME "ESP32"
#define BLYNK_AUTH_TOKEN "YourAuthToken"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

char ssid[] = "YOUR_WIFI_NAME";
char pass[] = "YOUR_WIFI_PASSWORD";

BlynkTimer timer;

void sendData() {
  int sensorValue = random(20,40);

  Blynk.virtualWrite(V0, sensorValue);

  Serial.print("Temperature: ");
  Serial.println(sensorValue);
}

void setup() {
  Serial.begin(115200);

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
```



```
    timer.setInterval(2000L, sendData);  
}
```

```
void loop() {  
    Blynk.run();  
    timer.run();  
}
```

Observation

- ESP32 successfully connected to Wi-Fi and cloud server.
- Sensor data was uploaded to the cloud dashboard.
- Real-time values were displayed on the mobile/web dashboard.

Result

Thus, cloud connectivity between ESP32 and IoT cloud platform was successfully established for remote monitoring and visualization.

Exp No 8: Database & Google Sheets Logging using ESP32

Aim

To store IoT sensor data using cloud databases and log ESP32 sensor readings into Google Sheets for monitoring and analysis.

Apparatus Required

- ESP32 Development Board
- USB Cable
- Wi-Fi Network
- Arduino IDE
- Google Account
- Google Sheets
- Firebase Account (optional)
- DHT11 Sensor (optional)

Theory

IoT systems continuously generate sensor data that must be stored and analyzed efficiently. Cloud databases such as Firebase and Google Sheets enable real-time logging, remote monitoring, and historical analysis of sensor data.

Firebase Features

- Real-time synchronization
- JSON-based NoSQL database
- REST API support

Google Sheets Logging

Google Sheets can be used as a lightweight cloud database for IoT projects. Data is sent through Google Apps Script using HTTP GET/POST requests.

Common logged fields:

- Timestamp
- Sensor value

- Device ID
- Status

Circuit Diagram

ESP32 connected to computer through USB cable and connected to Wi-Fi network.

Procedure

1. Create a Google Sheet.
2. Open Extensions → Apps Script.
3. Write Apps Script for data logging.
4. Deploy script as Web App.
5. Copy Web App URL.
6. Install required libraries in Arduino IDE.
7. Connect ESP32 to Wi-Fi.
8. Upload ESP32 code with Web App URL.
9. Observe sensor data updates in Google Sheets.

Google Apps Script

```
function doGet(e) {

    var sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();

    sheet.appendRow([
        new Date(),
        e.parameter.temperature
    ]);

    return ContentService.createTextOutput("Success");
}
```

ESP32 Program

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

String scriptURL = "YOUR_GOOGLE_SCRIPT_WEBAPP_URL";

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void loop() {

  if (WiFi.status() == WL_CONNECTED) {

    HTTPClient http;

    int temp = random(20,40);

    String url = scriptURL + "?temperature=" + String(temp);

    http.begin(url);

    int httpCode = http.GET();

    Serial.print("HTTP Response Code: ");
    Serial.println(httpCode);

    http.end();
  }
}
```

```
delay(5000);  
}
```

Observation

- ESP32 successfully connected to Wi-Fi.
- Sensor values were transmitted to Google Sheets.
- Timestamp and temperature values were logged correctly.
- Cloud-based data storage was achieved.

Result

Thus, ESP32 sensor data was successfully stored and logged using Google Sheets and cloud database concepts.

Exp No 9:

Multi-Node IoT Systems using ESP32

Aim

To study and implement a multi-node IoT system where multiple ESP32 nodes communicate with a central gateway/cloud platform for distributed sensing and monitoring.

Apparatus Required

- ESP32 Development Boards (2 or more)
- USB Cables
- Wi-Fi Network
- Arduino IDE
- MQTT Broker / Cloud Platform
- Sensors (DHT11, LDR, Soil Moisture, etc.)

Theory

A multi-node IoT system consists of several sensor or actuator nodes communicating with a central gateway or cloud platform. These systems improve scalability, reliability, and sensing coverage in smart applications.

Advantages

- Enhanced sensing coverage
- Reduced single-point failure
- Scalable architecture
- Real-time distributed monitoring

Common Topologies

1. Star Topology

All nodes communicate with a central gateway.

2. Mesh Topology

Nodes communicate with neighboring nodes for higher reliability.

3. Hybrid Topology

Combination of star and mesh networks.

Protocols Used

- MQTT
- HTTP/REST
- ESP-NOW
- LoRaWAN

Circuit Diagram

Multiple ESP32 nodes connected through Wi-Fi network and communicating with MQTT broker/cloud server.

Procedure

1. Configure Wi-Fi connectivity on all ESP32 nodes.
2. Install required MQTT libraries.
3. Configure MQTT broker details.
4. Assign separate topics for each node.
5. Upload sensor publishing program to all nodes.
6. Configure gateway/cloud dashboard to receive data.
7. Monitor incoming data from multiple nodes.

Program for Node 1

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);

void setup() {
```

```

Serial.begin(115200);

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
}

client.setServer(mqtt_server, 1883);
}

void reconnect() {
    while (!client.connected()) {
        client.connect("ESP32_Node1");
    }
}

void loop() {

    if (!client.connected()) {
        reconnect();
    }

    client.loop();

    int temp = random(20,35);

    client.publish("node1/temperature",
        String(temp).c_str());

    delay(3000);
}

```

Program for Node 2

```

#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

```



```
const char* mqtt_server = "broker.hivemq.com";
```

```
WiFiClient espClient;
```

```
PubSubClient client(espClient);
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    WiFi.begin(ssid, password);
```

```
    while (WiFi.status() != WL_CONNECTED) {
```

```
        delay(500);
```

```
    }
```

```
    client.setServer(mqtt_server, 1883);
```

```
}
```

```
void reconnect() {
```

```
    while (!client.connected()) {
```

```
        client.connect("ESP32_Node2");
```

```
    }
```

```
}
```

```
void loop() {
```

```
    if (!client.connected()) {
```

```
        reconnect();
```

```
    }
```

```
    client.loop();
```

```
    int humidity = random(40,80);
```

```
    client.publish("node2/humidity",
```

```
        String(humidity).c_str());
```

```
    delay(3000);
```

```
}
```

Observation

- Multiple ESP32 nodes successfully connected to Wi-Fi.
- Sensor data from different nodes was transmitted independently.
- MQTT broker/cloud dashboard received data from all nodes.
- Distributed monitoring was achieved successfully.

Result

Thus, a multi-node IoT system using ESP32 and MQTT communication was successfully implemented for distributed sensing and monitoring.

Exp No 10 :**IoT Security & Optimization using ESP32****Aim**

To study IoT security mechanisms and implement optimization techniques for secure and efficient ESP32-based IoT systems.

Apparatus Required

- ESP32 Development Board
- USB Cable
- Wi-Fi Network
- Arduino IDE
- MQTT Broker / Cloud Platform
- Sensors (optional)

Theory

IoT security protects devices, communication channels, and data from unauthorized access and cyber threats. Optimization improves device efficiency, reduces power consumption, and enhances system performance.

IoT Security Concepts

- Authentication
- Encryption
- Access control
- Secure communication
- Firmware security

Common Security Threats

- Unauthorized access
- Data leakage
- Weak authentication
- Unencrypted communication

Optimization Techniques

- Deep sleep mode
- Efficient data transmission
- Edge processing
- Duty cycling

Circuit Diagram

ESP32 connected to Wi-Fi network and cloud platform with secure MQTT communication.

Procedure

1. Install Arduino IDE and ESP32 board package.
2. Configure Wi-Fi credentials.
3. Enable secure MQTT communication.
4. Implement authentication and encrypted communication.
5. Add deep sleep optimization in ESP32 code.
6. Upload the program to ESP32.
7. Monitor secure data transmission and power optimization.

Program

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

const char* mqtt_server = "broker.hivemq.com";

WiFiClientSecure espClient;
PubSubClient client(espClient);
```

```

void setup_wifi() {

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void reconnect() {

  while (!client.connected()) {

    if (client.connect("ESP32SecureClient")) {
      client.publish("iot/security",
        "Secure Connection Established");
    }
    else {
      delay(5000);
    }
  }
}

void setup() {

  Serial.begin(115200);

  setup_wifi();

  espClient.setInsecure();

  client.setServer(mqtt_server, 8883);
}

void loop() {

  if (!client.connected()) {
    reconnect();
  }

  client.loop();
}

```

```
int sensorValue = random(20,40);

client.publish("iot/security",
              String(sensorValue).c_str());

Serial.println("Data Sent Securely");

Serial.println("Entering Deep Sleep");

esp_sleep_enable_timer_wakeup(10 * 1000000);

esp_deep_sleep_start();
}
```

Observation

- ESP32 successfully connected using secure MQTT communication.
- Data transmission occurred through encrypted connection.
- Deep sleep mode reduced power consumption.
- Reliable and optimized IoT communication was achieved.

Result

Thus, IoT security mechanisms and optimization techniques were successfully implemented using ESP32 for secure and energy-efficient IoT communication.

Exp No 11:

REAL-WORLD IoT APPLICATIONS

Aim

To design, implement, and analyze a real-world IoT application using sensors, ESP32 microcontroller, cloud connectivity, and automated control mechanisms.

Objectives

- Understand the architecture of real-world IoT systems.
- Interface sensors with ESP32.
- Collect and transmit sensor data.
- Implement automation logic based on sensor values.
- Analyze deployment challenges and scalability.

Theory

Internet of Things (IoT) applications are widely used in smart homes, agriculture, healthcare, transportation, and industries. Real-world IoT systems integrate sensors, controllers, cloud platforms, and communication networks to automate monitoring and control operations.

A typical IoT system contains:

1. Sensors for data acquisition.
2. Microcontroller for processing.
3. Communication module for cloud connectivity.
4. Cloud dashboard for visualization.
5. Actuators for automation.

This experiment demonstrates a practical IoT application using environmental monitoring and automated control.

Components Required

S.No	Component	Quantity
1	ESP32 Development Board	1
2	Soil Moisture Sensor / DHT11 Sensor	1
3	Relay Module	1
4	Water Pump / LED	1
5	Jumper Wires	As required
6	Breadboard	1
7	USB Cable	1
8	Wi-Fi Network	1

Block Diagram

Sensor → ESP32 Controller → Wi-Fi/Cloud → Dashboard ↓ Relay Control ↓ Pump / Appliance

Circuit Connections

Soil Moisture Sensor to ESP32

Sensor Pin ESP32 Pin

VCC 3.3V

GND GND

AO GPIO 34

Relay Module

Relay Pin ESP32 Pin

IN GPIO 26

VCC 5V

GND GND

Procedure

1. Connect the sensor and relay module to the ESP32.
2. Install Arduino IDE and ESP32 board package.
3. Write and upload the IoT monitoring code.
4. Connect ESP32 to Wi-Fi.
5. Monitor sensor data through Serial Monitor or cloud dashboard.
6. Observe automated switching based on threshold values.
7. Record sensor readings and actuator response.

Arduino Code

```
#define sensorPin 34
```

```
#define relayPin 26
```

```
int threshold = 2000;
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  pinMode(relayPin, OUTPUT);
```

```
}
```

```
void loop() {
```

```
  int sensorValue = analogRead(sensorPin);
```

```
  Serial.print("Moisture Value: ");
```

```
  Serial.println(sensorValue);
```

```
  if(sensorValue > threshold) {
```

```
    digitalWrite(relayPin, HIGH);
```

```
    Serial.println("Pump ON");
```

```
}  
else {  
    digitalWrite(relayPin, LOW);  
    Serial.println("Pump OFF");  
}  
  
delay(2000);  
}
```

Observation Table

Trial Sensor Reading Threshold Relay Status

1

2

3

4

Applications

- Smart Irrigation Systems
- Smart Home Automation
- Industrial Monitoring
- Environmental Monitoring
- Energy Management

Advantages

- Reduces manual intervention.
- Enables remote monitoring.
- Improves efficiency and reliability.

- Supports real-time automation.

Deployment Challenges

- Network instability
- Sensor degradation
- Power management issues
- Cybersecurity threats
- Environmental interference

Result

The real-world IoT application using ESP32 was successfully implemented. Sensor data was monitored in real time, and automated control actions were performed based on predefined threshold conditions.

Viva Questions

1. What is IoT?
2. Why is ESP32 preferred in IoT applications?
3. What is the purpose of a relay module?
4. What are the challenges in IoT deployment?
5. Explain the role of cloud connectivity in IoT.
6. What are the advantages of automated irrigation?

Aim

To design, implement, test, and evaluate a complete IoT system using ESP32, sensors, cloud connectivity, and automation techniques.

Objectives

- Develop a real-world IoT prototype.
- Integrate sensors, actuators, and cloud platforms.
- Implement wireless communication protocols.
- Analyze system performance and reliability.
- Evaluate security and scalability of the IoT solution.

Theory

The final IoT project combines embedded systems, networking, cloud computing, and automation into a single functional application. IoT systems collect real-time data through sensors, process it using microcontrollers such as ESP32, and transmit the information to cloud platforms for monitoring and control.

A complete IoT system generally includes:

1. Physical sensing units.
2. Embedded processing hardware.
3. Wireless communication layer.
4. Cloud dashboard and data storage.
5. User interface and automation controls.

This experiment demonstrates the full development cycle of an IoT application from requirement analysis to testing and evaluation.

Components Required

S.No	Component	Quantity
1	ESP32 Development Board	1
2	DHT11 / Soil Moisture / Gas Sensor	1
3	Relay Module	1
4	LED / Water Pump / Buzzer	1
5	Breadboard	1
6	Jumper Wires	As required
7	Wi-Fi Connection	1
8	Cloud Platform (Blynk/ThingSpeak)	1

Project Theme Example

Smart Agriculture Monitoring System

The system monitors soil moisture and environmental conditions, uploads data to the cloud, and automatically controls irrigation.

System Architecture

Sensor Layer → ESP32 Controller → Wi-Fi Network → Cloud Dashboard ↓ Relay Control ↓ Water Pump

Hardware Connections

Sensor Connections

Sensor Pin ESP32 Pin

VCC 3.3V

GND GND

DATA/AO GPIO 34

Relay Connections

Relay Pin ESP32 Pin

IN GPIO 26

VCC 5V

GND GND

Software Requirements

- Arduino IDE
- ESP32 Board Package
- Wi-Fi Library
- Blynk / ThingSpeak Library
- USB Drivers

Procedure

1. Define the project objective and requirements.
2. Design the hardware architecture.
3. Connect sensors and actuators to ESP32.
4. Configure Wi-Fi communication.
5. Develop cloud dashboard for monitoring.
6. Write and upload embedded code.
7. Test sensor readings and automation.
8. Validate cloud communication.
9. Analyze performance and reliability.
10. Document results and observations.

Sample Arduino Code

```
#define sensorPin 34

#define relayPin 26


int threshold = 2000;


void setup() {
  Serial.begin(115200);
  pinMode(relayPin, OUTPUT);
}


void loop() {
  int sensorValue = analogRead(sensorPin);


  Serial.print("Sensor Value: ");
  Serial.println(sensorValue);


  if(sensorValue > threshold) {
    digitalWrite(relayPin, HIGH);
    Serial.println("Device ON");
  }
  else {
    digitalWrite(relayPin, LOW);
    Serial.println("Device OFF");
  }


  delay(2000);
}
```

Testing Parameters

Parameter	Observation
-----------	-------------

Sensor Accuracy	
-----------------	--

Wi-Fi Connectivity	
--------------------	--

Dashboard Response	
--------------------	--

Automation Accuracy	
---------------------	--

System Reliability	
--------------------	--

Performance Evaluation

The following aspects must be evaluated:

- Data transmission latency
- Sensor accuracy
- Cloud response time
- Power consumption
- Communication reliability

Security Validation

- Protect API keys and credentials.
- Use authenticated cloud access.
- Ensure secure Wi-Fi communication.
- Prevent unauthorized device control.

Applications

- Smart Home Automation
- Smart Agriculture
- Industrial IoT Monitoring

- Environmental Monitoring
- Healthcare Monitoring

Advantages

- Real-time monitoring and control.
- Remote accessibility.
- Improved operational efficiency.
- Reduced manual effort.
- Better decision-making using cloud analytics.

Challenges Faced

- Network instability
- Sensor calibration issues
- Power management problems
- Security vulnerabilities
- Environmental interference

Result

The ESP32-based IoT project was successfully designed, implemented, and evaluated. The system demonstrated real-time monitoring, wireless communication, cloud integration, and automated control operations.

Viva Questions

1. What are the major components of an IoT system?
2. Why is ESP32 widely used in IoT projects?
3. Explain the role of cloud dashboards.
4. What communication protocols are commonly used in IoT?
5. What are the challenges in IoT deployment?
6. How can IoT systems be secured?